

互動式網頁設計-說明文件

組員：

CBF111063 黃筠晴

CBF111064 陳思誠

CBF111066 林展德

功能說明

首頁設計

本專題首頁採用深色科技風格設計，作為整個網站的入口頁面。首頁包含專題標題、導覽列、主視覺區塊、活動廣告區、遊戲介紹、操作說明、遊戲規則及角色預覽等內容，讓使用者能快速了解遊戲玩法與專題特色。

翻牌配對遊戲

玩家需要翻開卡牌並記住圖片位置，找出相同圖片完成配對。成功配對即可獲得分數，若翻開不同圖片則卡牌會自動翻回。

AI 對戰模式

遊戲加入 AI 玩家機制，系統會模擬 AI 的記憶能力與玩家進行競賽。透過比較雙方配對數量決定勝負，提高遊戲挑戰性。

難度選擇功能

遊戲提供簡單模式與困難模式兩種難度：

簡單模式：AI 記憶 3 張牌的位置。

困難模式：AI 記憶 5 張牌的位置。

60 秒極限挑戰模式

玩家需在 60 秒內完成最多配對，系統會即時計算分數與配對數量，增加遊戲刺激感與競爭性。

排行榜功能

系統會紀錄玩家的挑戰成績，並依照分數進行排序，讓玩家可以比較彼此的表現。

角色介紹功能

首頁提供咖波、狗狗及兔兔等角色介紹，並可透過角色頁面瀏覽更完整的角色資訊，增加遊戲趣味性與主題特色。

活動資訊展示

首頁側邊廣告區提供咖波 10 週年特展資訊，並可連結至官方活動網站，讓使用者獲得更多相關資訊。

程式碼介紹

首頁(home.html)

程式碼重點-導覽列

```
<nav class="full-nav">
  <a href="#intro">🎮 遊戲介紹</a>
  <a href="#features">🔧 操作說明</a>
  <a href="#rules">📖 遊戲規則</a>
  <a href="#characters">🐾 角色預覽</a>
</nav>
```

說明

- 使用 HTML 建立導覽列
- 利用錨點連結快速跳轉到指定區塊
- 提升網站瀏覽便利性

CSS (style.css)

程式碼重點-首頁大咖波圖

```
.big-card{
  width:260px;
  height:260px;
  border-radius:50%;
  background:radial-gradient(circle, #ffffff 0%, #ffffff 58%, #00A2E8 59%, #0077b3 100%);
  display:flex;
  justify-content:center;
  align-items:center;
  overflow:hidden;
  box-shadow:
    0 0 25px rgba(0,162,232,.9),
    0 0 60px rgba(0,162,232,.45);
  animation:capooFloat 3s infinite ease-in-out;
}

@keyframes capooFloat{
  0%{ transform:translateY(0); }
  50%{ transform:translateY(-15px); }
  100%{ transform:translateY(0); }
}
```

說明

- animation: 啟用 CSS 動畫功能
- @keyframes capooFloat{
 0%{ transform:translateY(0); }
 50%{ transform:translateY(-15px); }
 100%{ transform:translateY(0); }

0% 在原本位置 50% 往上飄 15px 100% 回到原本位置 所以咖波會上下浮動。

- Infinite:無限循環

AI 記憶對戰模式 (game.html)

AI 記憶佇列

```
function addQueue(card) {
```

```
    if(aiQueue.some(item => item.card === card)) return; → 防止重複記錄
```

```
    aiQueue.push({ card: card, img: $(card).data("img") });
```

```
    → 將卡片加入記憶佇列
```

```
    let el = $(`<div class="q-item q-add"></div>`);
```

```
    $("#ai-queue").append(el); → 顯示在畫面上
```

```
    if(aiQueue.length > MAX_Q) { aiQueue.shift(); } → 超過容量則移除最舊的
```

```
}
```

```
function findMatch() {
```

```
    for(let i=0; i<aiQueue.length; i++) {
```

```
        for(let j=i+1; j<aiQueue.length; j++) {
```

```
            if(aiQueue[i].img === aiQueue[j].img && aiQueue[i].card !== aiQueue[j].card) {
```

```
                return aiQueue[i].img; → 找到相同圖片的兩張不同卡片
```

```
            }
```

```
        }
```

```
    }
```

```
    return null; → 沒找到
```

```
}
```

AI 對戰模式 (index.html)

卡牌配對邏輯 function checkPlayer(second){

lock = true; → 鎖定防止連點

if(\$(pFirst).data("img") === \$(second).data("img")) {

//  配對成功

\$(pFirst).add(second).addClass("done"); → 兩張牌移除

pScore++; → 加分

pCombo++; → Combo +1

aiCombo = 0; → 中斷對方 Combo

if(pCombo >= 2) {

freezeAI = true; → 凍結 AI

\$("#freeze-ai").fadeIn(200); → 顯示凍結特效

setTimeout(() => { freezeAI = false; \$("#freeze-ai").fadeOut(200); }, 3000);

pCombo = 0;

}

checkGameOver(); → 檢查是否獲勝

}

else {

//  配對失敗

pCombo = 0;

setTimeout(() => { \$(pFirst).add(second).removeClass("flipped"); }, 700);

→ 0.7 秒後翻回背面

```
}
```

```
setTimeout(() => { pFirst = null; lock = false; }, 700);
```

```
}
```

60 秒極限挑戰模式（challenge.html）

```
function saveAndRedirect() {  
  // 🌀 核心確保：把當前畫面上最新、最即時的數據存進去  
  localStorage.setItem('gameMode', 'challenge');  
  localStorage.setItem('challengeScore', matchedPairs); // 儲存實際配對組數  
  localStorage.setItem('challengeTimeLeft', timeLeft < 0 ? 0 : timeLeft); // 確保時間不為負數  
  localStorage.setItem('challengeScoreProcessed', 'false'); // 開啟排行榜寫入訊號  
  
  // 順利跳轉到結算頁  
  window.location.href = "result.html";  
}  
  
function resetChallengeGame() {  
  // 1. 清除舊的計時器，防止倒數速度加倍  
  clearInterval(timerInterval);  
  
  // 2. 數據全部回歸初始值  
  timeLeft = 60;  
  matchedPairs = 0;  
  flippedCards = [];  
  isLockBoard = false;  
  
  // 3. 更新畫面上的文字  
  $('#timer').text(timeLeft);  
  $('#score').text(matchedPairs);  
  
  // 4. 解除排行榜的已處理標記，確保重玩這局的分數等一下能順利送進排行榜  
  localStorage.setItem('challengeScoreProcessed', 'false');  
  
  // 5. 重新洗牌並生成大棋盤、重新啟動倒數計時  
  initChallengeBoard();  
  startTimer();  
}
```

```
window.onload = function() {  
  // 🌀 1. 核心核心：先去網址列把 ?user= 後面的暱稱挖出來！  
  let urlParams = new URLSearchParams(window.location.search);  
  let pNickname = urlParams.get('user') || localStorage.getItem('playerNickname') || '挑戰玩家';  
  
  // 🌀 2. 同步把暱稱存進 localStorage，這樣等一下排行榜（leaderboard.html）才能跨頁抓到！  
  localStorage.setItem('playerNickname', pNickname);  
  
  // 🌀 3. 華麗地渲染到最頂端的「挑戰玩家」畫面上  
  $('#player-nickname-display').text(pNickname);  
  
  // 4. 隨機抽選出賽英雄（保持原樣）  
  const heroPool = ['貓貓蟲卡波', '狗狗', '咪卡', '兔兔', '黑貓蟲', '奶泡貓', '暴龍姐'];  
  let randomIndex = Math.floor(Math.random() * heroPool.length);  
  let randomHero = heroPool[randomIndex];  
  localStorage.setItem('playerCharName', randomHero);  
  $('#hero-display').text(randomHero);  
  
  // 5. 加上重置訊號，啟動遊戲  
  localStorage.setItem('challengeScoreProcessed', 'false');  
  initChallengeBoard();  
  startTimer();  
};
```

戰果結果(result.html)

程式碼重點

```
<script>
window.onload = function() {
    renderLeaderboard();
};

function renderLeaderboard() {
    let $tbody = $('#leaderboard-body').empty();

    //統一使用 'challengeLeaderboard' 作為緩存金鑰
    let records = JSON.parse(localStorage.getItem('challengeLeaderboard')) || [];
    let latestHero = localStorage.getItem('playerCharName') || '貓貓蟲卡波';

    let urlParams = new URLSearchParams(window.location.search);
    let latestNickname = localStorage.getItem('playerNickname') || urlParams.get('user') || '挑戰玩家';

    let latestScore = parseInt(localStorage.getItem('challengeScore'));
    let latestTimeLeft = parseInt(localStorage.getItem('challengeTimeLeft'));

    if (!isNaN(latestScore) && !isNaN(latestTimeLeft)) {
        let isScoreProcessed = localStorage.getItem('challengeScoreProcessed') === 'true';

        if (!isScoreProcessed) {
            records.forEach(r => r.isLatest = false);

            // 將網址撈到的正確暱稱當成獨立屬性存進排行資料庫
            records.push({
                hero: latestHero,
                nickname: latestNickname,
                score: latestScore,
                timeLeft: latestTimeLeft,
                isLatest: true
            });

            localStorage.setItem('challengeScoreProcessed', 'true');
            localStorage.removeItem('challengeScore');
        }
    }
}
```

說明

- 使用 localStorage 儲存玩家資料
- 讀取暱稱、分數與剩餘時間
- 當遊戲結束後，系統會自動載入玩家成績並完成結果顯示。

排行榜(leaderboard.html)

```
function renderLeaderboard() {
  let $tbody = $('#leaderboard-body').empty();

  // 核心：統一使用 'challengeLeaderboard' 作為緩存金鑰
  let records = JSON.parse(localStorage.getItem('challengeLeaderboard')) || [];

  // 1. 抓取剛才從挑戰賽或結算頁面傳過來的「最新一筆戰果」
  let latestHero = localStorage.getItem('playerCharName') || '貓貓龜卡波';

  // 攔截玩家在輸入頁傳過來的參數
  let urlParams = new URLSearchParams(window.location.search);
  let latestNickname = localStorage.getItem('playerNickname') || urlParams.get('user') || '挑戰玩家';

  let latestScore = parseInt(localStorage.getItem('challengeScore'));
  let latestTimeLeft = parseInt(localStorage.getItem('challengeTimeLeft'));

  if (!isNaN(latestScore) && !isNaN(latestTimeLeft)) {
    let isScoreProcessed = localStorage.getItem('challengeScoreProcessed') === 'true';

    if (!isScoreProcessed) {
      records.forEach(r => r.isLatest = false);

      // 將網址撈到的正確暱稱當成獨立屬性存進排行資料庫
      records.push({
        hero: latestHero,
        nickname: latestNickname,
        score: latestScore,
        timeLeft: latestTimeLeft,
        isLatest: true
      });
    }
  }
}
```

/* 🏆 排行榜大卡片外框 */

```
.leaderboard-box {
  background-color: #1f1f23;
  border: 2px solid #333;
  border-radius: 20px;
  padding: 40px;
  width: 100%;
  max-width: 650px; /* 配合大棋盤的 650px 黃金寬度 */
  box-shadow: 0 10px 30px rgba(0,0,0,0.6);
  margin-top: 20px;
}
```

```

/* 前三名華麗稱號特效 */
tr.rank-1 td { color: var(--gold); font-weight: bold; }
tr.rank-2 td { color: var(--silver); font-weight: bold; }
tr.rank-3 td { color: var(--bronze); font-weight: bold; }

.medal {
  font-size: 18px;
  margin-right: 5px;
}

/* 最新戰果提示標籤 */
.latest-tag {
  background-color: var(--accent-color);
  color: white;
  font-size: 11px;
  padding: 2px 6px;
  border-radius: 4px;
  margin-left: 8px;
  font-weight: bold;
  vertical-align: middle;
}

```

```

// 🗑 清除/重置排行榜功能（徹底歸零版）
function clearLeaderboard() {
  if (confirm("確定要重設排行榜，將所有數據徹底歸零嗎？")) {
    // 1. 🐛 關鍵Bug修正：將名稱對準 'challengeLeaderboard'，存入空陣列
    localStorage.setItem('challengeLeaderboard', JSON.stringify([]));

    // 2. 同步清空當前剛玩完的挑戰賽暫存最新記錄
    localStorage.removeItem('challengeScore');
    localStorage.removeItem('challengeTimeLeft');
    localStorage.setItem('challengeScoreProcessed', 'false');

    // 3. 重新渲染畫面（這時表格就會完全清空歸零了）
    renderLeaderboard();
  }
}

```

角色介紹(story.html)

```
<html lang="zh-Hant"> #設定網頁語言為繁體中文
```

```
<head>
```

```
<meta charset="UTF-8"> #設定網頁編碼為 UTF-8，確保中文字不會變成亂碼。
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

讓網頁在手機螢幕上也能正常顯示，不會縮得太小。

#width=device-width：它告訴瀏覽器：「請把網頁的排版寬度，設定成跟設備螢幕的實體寬度一模一樣」。如果沒有這句，手機瀏覽器會預設這是一個傳統的電腦版網頁（通常寬度會預設在 980px 左右），然後硬生生地把整個畫面「拉遠」縮得很小，導致字小到看不見。

#initial-scale=1.0：這代表畫面一開始載入時的「縮放比例是 100%」（也就是 1.0 倍），確保畫面呈現最原始、不失真的大小，不會預設被放大或縮小。

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

#載入 jQuery 函式庫，方便後面寫 JavaScript 控制畫面。

```
body{
```

```
background-color: var(--bg-color); /* 使用背景色變數 */
```

```
color: var(--text-color); /* 使用文字色變數 */
```

```
font-family: 'Segoe UI', Roboto, sans-serif; /* 設定英中文字型順序 */
```

```
margin: 0; /* 清除瀏覽器預設外邊距 */
```

```
padding: 20px; /* 網頁四周內邊距 */
```

```
padding-top: 80px; /* 頂部留空，防止內容被固定導覽列遮住 */
```

```
display: flex; /* 啟用 Flexbox 排版，讓網頁裡的所有東西都垂直置  
中排列 */
```

```
flex-direction: column; /* 讓內容由上往下垂直排列 */
```

```
align-items: center; /* 水平置中對齊所有元件 */
```

```
overflow-x: hidden; /* 隱藏水平滾動條，防止 3D 卡片轉到外側時撐開網  
頁 */ }
```

```
h2{
```

```
color: var(--accent-color);
```

```
text-shadow: 0 0 10px rgba(77,166,255,0.3);
```

```
/* 為標題加上淡淡的亮藍色霓虹發光效果(原本是做「文字陰影」的屬性) */
```

- 0 0：代表陰影在水平和垂直方向都不偏移，正正地疊在文字正後方。
- 10px：代表陰影的「模糊程度」，數值越大，光暈散得越開。
- rgba(77,166,255,0.3)：這是陰影的顏色，對應的是前面那個亮藍色的 RGB 色碼，最後的 0.3 代表 30% 的透明度。綜合起來，就會在字體背後形成一層淡淡的藍色光暈！

```
margin-bottom: 5px;
```

```
/* 3D 舞台的核心外框 */
```

```
.stage-3d{
```

```
width: 300px;
```

```
height: 380px;
```

```
perspective: 2000px; /* 關鍵：設定 3D 透視深度。數值越小近大遠小越明顯。這裡設 2000px 可以給予卡片舒適的立體空間感，避免卡片離鏡頭太近被裁剪 */
```

```
margin: 30px auto 10px; /* CSS 中 margin（外邊距）的縮寫寫法，代表的順序是「上、左右、下」 */
```

```
position: relative; /* （絕對定位）在舞台周圍拉起一條隱形的封鎖線，告訴裡面的 3D 卡片：「你們的旋轉和絕對定位，都必須以我這個框框為基準，不准跑出去！」 */
```

```
/* 旋轉木馬主體 */
```

```
.carousel-3d {
```

```
width: 100%; height: 100%; /*填滿舞台*/
```

```
position: absolute; /*絕對定位*/
```

2. position: absolute; (絕對定位)

搭配外層舞台的 `position: relative;`，這行讓 `.carousel-3d` 精準地貼合在舞台的左上角（原點），並覆蓋在整個舞台的範圍內。有了這個設定，不管裡面的卡片怎麼轉，轉盤本身的軸心都會穩穩地固定在舞台正中央。

```
transform-style: preserve-3d;
```

/*關鍵：宣告這個子元素內部要啟用「真實 3D 空間」渲染，否則卡片會變成平面的 */

```
transition: transform 0.8s cubic-bezier(0.175, 0.885, 0.32, 1.15); /* 當旋轉木馬轉動時，觸發  
0.8 秒的平滑動畫，cubic-bezier 帶有一點點帥氣的「回彈 (Overshoot)」效果 */  
/*貝茲曲線*/
```

```
}
```

```
/* 角色卡牌基礎設定 */
```

```
.character-card-3d {
```

```
position: absolute;
```

```
width: 160px; /* 預設沒被選中時，呈現圓形大頭貼的寬 */
```

```
height: 160px; /* 預設高 */
```

/* 以下四行是極為精準的 3D 置中密技：先推到父容器正中央，再用負邊距(Margin)往回拉自身寬高的一半，確保旋轉時的圓心是絕對完美的 */

```
left: 50%; top: 50%;
```

```
margin-left: -80px;
```

```
margin-top: -80px;
```

`margin-left: -80px; margin-top: -80px;`：這是經典的「負邊距置中法」！因為卡片寬高都是 160px，所以我們利用負邊距，把卡片往左、往上各自拉回一半的距離 (80px)。這樣一來，卡片的「正中心點」就會完美對齊轉盤的「正中心點」，這是確保 3D 旋轉軌道呈現完美圓形的關鍵。

`border-radius: 50%;` /*把原本方形的卡片削成完美的圓形，這就是為什麼一開始每個角色看起來都是圓形大頭貼*/

`padding: 10px;` /*讓裡面的圖片跟卡片邊緣保持 10 像素的內距，才不會貼得太死*/

`display: flex; # Flexbox` 內部元素排版 (讓圖片文字乖乖待在中間)

`flex-direction: column;`

`align-items: center;`

`justify-content: center;`

`box-shadow: 0 10px 25px rgba(0,0,0,0.5);` /* 卡片下方的立體陰影 */

/*當卡片被點擊、寬高改變或變形時，以下所有屬性都會在 0.4 秒內平滑漸變*/

`transition: transform (3D 空間移動) 0.4s cubic-bezier(0.25, 1, 0.5, 1);`

/*一種貝茲曲線，這組參數會讓卡片彈出時有「快速衝出，然後平穩煞車」的視覺滑順感*/

/*當卡片被點擊、寬高改變或變形時，以下所有屬性都會在 0.4 秒內平滑漸變*/

`transition: transform (3D 空間移動) 0.4s cubic-bezier(0.25, 1, 0.5, 1);`

/*一種貝茲曲線，這組參數會讓卡片彈出時有「快速衝出，然後平穩煞車」的視覺滑順感*/

`border-color (邊框顏色漸變) 0.4s, opacity 0.4s, background-color 0.4s,`

`width 0.4s, height 0.4s, border-radius (從圓形變方形) 0.4s, margin-left 0.4s,`

`margin-top 0.4s;`

`opacity: 0.25;` /* 預設非焦點的角色會變得很透明，凸顯主角 */ (重新定位中心點)

`cursor: pointer;`

/* 當滑鼠游標移到卡片上時，游標會變成「手指圖案」，提示使用者這個圓形頭貼是可以點擊互動的*/

}

```
.character-info-content {  
  opacity: 0;          /* 預設完全透明看不見 */  
  max-height: 0;       /* 預設高度為 0 */  
  overflow: hidden; /* 超出範圍的文字隱藏 */  
  transition: opacity 0.2s ease-out, max-height 0.4s ease-out, margin-top 0.4s;
```

1. opacity 0.2s ease-out (透明度：快速淡入)

- **作用**：控制文字的透明度（從完全透明 0 變成完全不透明 1）。
- **時間 0.2s**：只花 0.2 秒。文字的「淡入」刻意設定得比卡片展開（0.4秒）還要快一點，這樣使用者可以第一時間看到文字浮現，而不需要等整個卡片長大完才開始看。
- **ease-out（減速效果）**：這是一種動畫節奏，意思是「一開始快，快結束時慢下來」。這會讓文字出現的感覺很輕柔，不會很生硬地「啪」一下彈出來。

```
width: 100%;  
text-align: center;  
}
```

// 點擊卡片本身的事件處理

```
$('.character-card-3d').click(function) {  
  let clickedIdx = $(this).data('idx'); // 獲取被點擊卡片的索引值  
  
  // 💡 防呆機制：只有當使用者點擊「目前正中央那張卡片」時，才可以展開  
  if (clickedIdx === currentIndex) {  
    // 如果這張卡片已經是打開狀態 (有 pop-out 類別)  
    if ($(this).hasClass('pop-out')) {  
      $(this).removeClass('pop-out'); // 移除打開狀態（變回圓形）  
      // 把卡片退回到圓環的正常半徑軌道上 (450px)  
      $(this).css('transform', `rotateY(${clickedIdx * rotateAngle}deg)  
        translateZ(${radiusZ}px)`);  
    }  
  }  
}
```

// 更新每張卡片狀態的函式

```
function updateCardStates() {  
  $('.character-card-3d').each(function) {  
    let idx = $(this).data('idx');  
    $(this).removeClass('active-char pop-out'); // 先清除所有人身上的焦點與展開樣式  
    // 把所有人的 3D 位置校正回標準的圓環軌道半徑 (450px)  
    $(this).css('transform', `rotateY(${idx * rotateAngle}deg) translateZ(${radiusZ}px)`);  
  });  
  // 單獨為目前正中央的那張卡片加上「active-char」亮起邊框與提高透明度  
  $('#char-${currentIndex}') .addClass('active-char');  
}
```

// 控制 3D 圓環主體旋轉的函式

```
function updateCarousel() {  
  // 關鍵原理：要讓第 currentIndex 張卡片轉到正前方，整個大圓環就必須往反方向轉過去！  
  // 所以角度要乘上負號 (-)  
  let targetDegree = currentIndex * -rotateAngle;  
  // 透過改變父容器的 rotateY，帶動裡面所有卡片一起像旋轉木馬般轉動  
  $('#carousel').css('transform', `rotateY(${targetDegree}deg)`);  
  updateCardStates(); // 每次轉完都要更新誰是目前的主角  
}
```

```
window.onload = function() {
```

```
  let $carousel = $('#carousel'); // 取得裝卡片的 3D 圓環容器
```

```
  // 使用 forEach 迴圈，把 characters 陣列裡的每隻角色依序拿出來生出 HTML
```

```
  characters.forEach((char, i) => {
```

```
    let cardHtml = ` //用字串模板把 HTML 寫好
```

```
let currentIndex = 0; // 紀錄目前網頁正中央顯示的是第幾個角色（從 0 開始算）
```

```
const rotateAngle = 360 / characters.length; // 算出一張卡片該旋轉的角度：360 度 / 15 張貼紙  
= 24 度
```

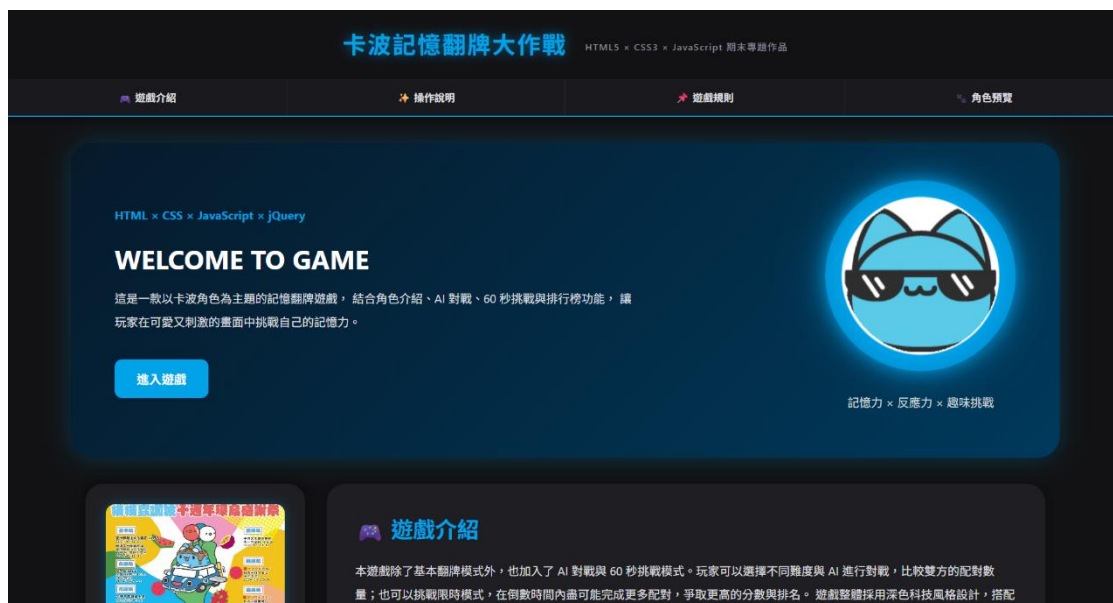
`characters.length` 會自動去數你的陣列裡面總共有幾個角色。以你的程式碼來說，總共有 15 個角色。

```
const radiusZ = 450; // 核心設定：3D 圓環的半徑。卡片會從中心點往外推 450px 圍成一個圓圈
```

```
    } else {  
        // 如果是關閉狀態  
        $('.character-card-3d').removeClass('pop-out'); // 先把其他可能開著的卡片全部  
        關掉  
        updateCardStates(); // 重置所有卡片到標準圓環位置  
          
        $(this).addClass('pop-out'); // 為自己加上 pop-out 類別（觸發 CSS 變方變大  
        動畫）  
        // 🚀 3D 爆衝視覺：除了旋轉原本角度外，把 Z 軸距離從 450px 猛推到  
        580px！讓卡片有「往螢幕前方衝出來」的超強立體感！  
        $(this).css('transform', `rotateY(${clickedIdx * rotateAngle}deg)  
        translateZ(580px)`);  
          
        // 點擊「◀ 上一個角色」按鈕時  
        $('#prev-btn').click(() => {  
            // 索引值減 1。為了防止變成負數，加上總長度後再取總長度的餘數（數學上的循環公  
            式）  
            currentIndex = (currentIndex - 1 + characters.length) % characters.length;  
            updateCarousel(); // 執行旋轉木馬轉動  
        });
```

執行畫面

首頁





SIDE AD

咖啡 10 週年特展

展覽日期
2024/06/29 ~ 2024/09/15

展覽地點
華山1914文創園區 東2A、東2B館

開放時間
10:00 AM ~ 06:00 PM

[更多資訊](#)

遊戲介紹

本遊戲除了基本翻牌模式外，也加入了 AI 對戰與 60 秒挑戰模式。玩家可以選擇不同難度與 AI 進行對戰，比較雙方的配對數量；也可以挑戰限時模式，在倒數時間內盡可能完成更多配對，爭取更高的分數與排名。遊戲整體採用深色科技風格設計，搭配咖啡家族角色圖片、動畫效果與排行榜系統，希望讓玩家在輕鬆有趣的環境中享受翻牌遊戲帶來的樂趣與挑戰。

翻牌配對

玩家一次翻開兩張牌，找出相同圖片即可成功配對。

AI 對戰

加入 AI 對手設定，讓玩家可以進行人機對戰。

60 秒挑戰

在限定時間內完成最多配對，增加緊張感。

排行榜

紀錄玩家挑戰成績，呈現排名與分數。

角色介紹預覽



SIDE AD

咖啡 10 週年特展

展覽日期
2024/06/29 ~ 2024/09/15

展覽地點
華山1914文創園區 東2A、東2B館

開放時間
10:00 AM ~ 06:00 PM

[更多資訊](#)

角色介紹預覽

首頁先展示部分角色，完整角色介紹可由上方導覽列進入。



咖啡

主角角色，外型可愛又充滿活力，是遊戲的代表角色。



狗狗

溫和又可愛的角色，在卡波世界中有重要的位置。



兔兔

可愛的角色之一，讓整體遊戲畫面更活潑。

遊戲規則

- 玩家每次可以翻開兩張卡牌。
- 如果兩張圖片相同，就成功配對並獲得分數。
- 如果圖片不同，卡牌會自動翻回去。
- 60 秒挑戰模式會計算成功配對組數與剩餘時間。
- 排行榜會依照挑戰結果進行排序。



SIDE AD

咖啡 10 週年特展

展覽日期
2024/06/29 ~ 2024/09/15

展覽地點
華山1914文創園區 東2A、東2B館

開放時間
10:00 AM ~ 06:00 PM

[更多資訊](#)

HTML5

負責網頁架構、內容區塊與頁面排版。

CSS3

負責深色主題、動畫、卡片效果與響應式設計。

JavaScript

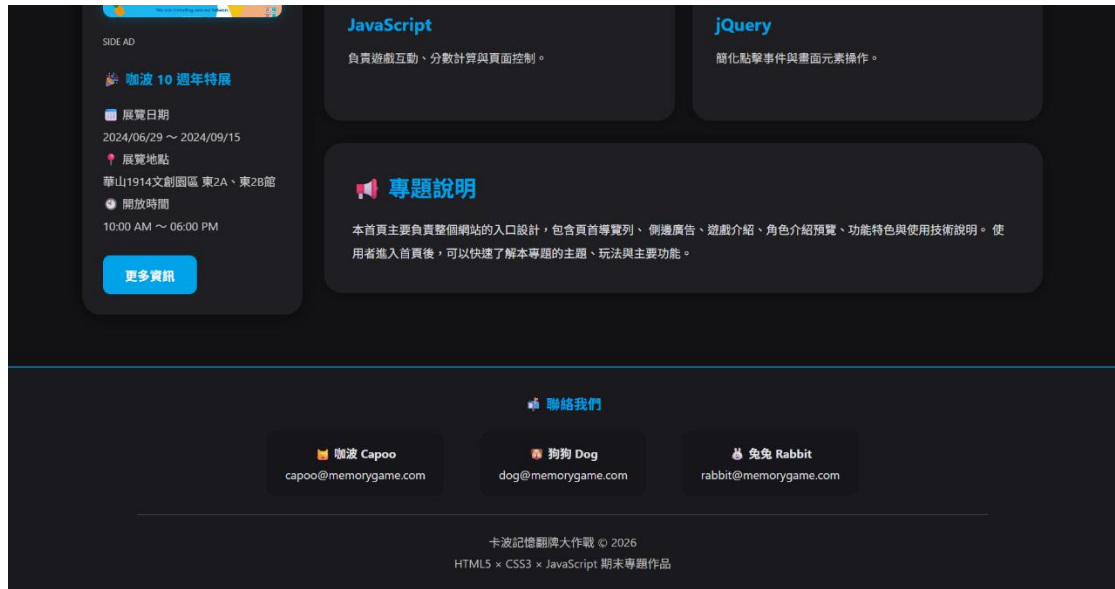
負責遊戲互動、分數計算與頁面控制。

jQuery

簡化點擊事件與畫面元素操作。

專題說明

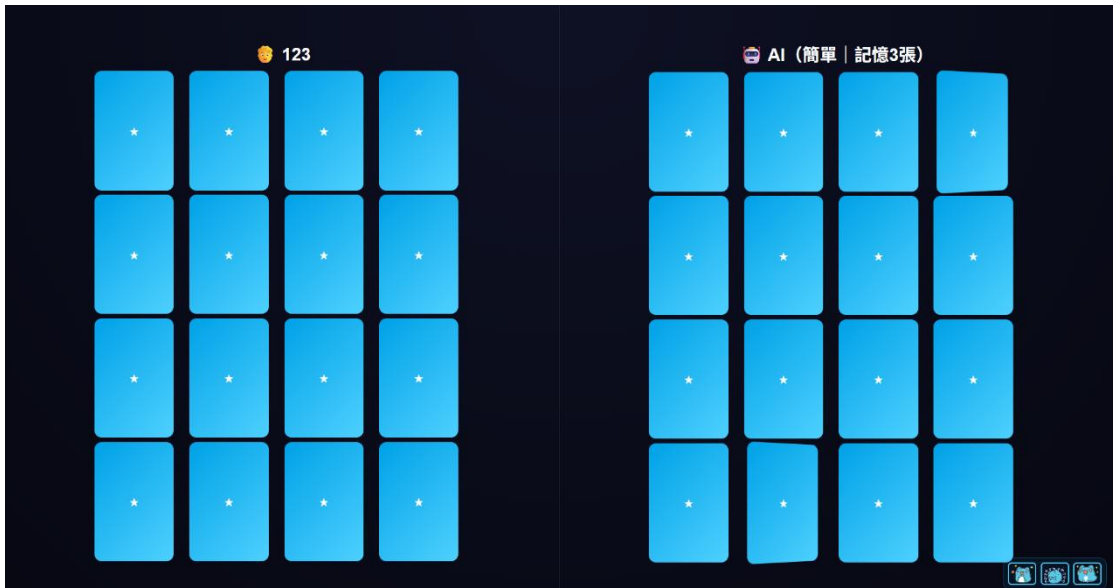
本首頁主要負責整個網站的入口設計，包含頁首導覽列、側邊廣告、遊戲介紹、角色介紹預覽、功能特色與使用技術說明。使用者進入首頁後，可以快速了解本專題的主題、玩法與主要功能。



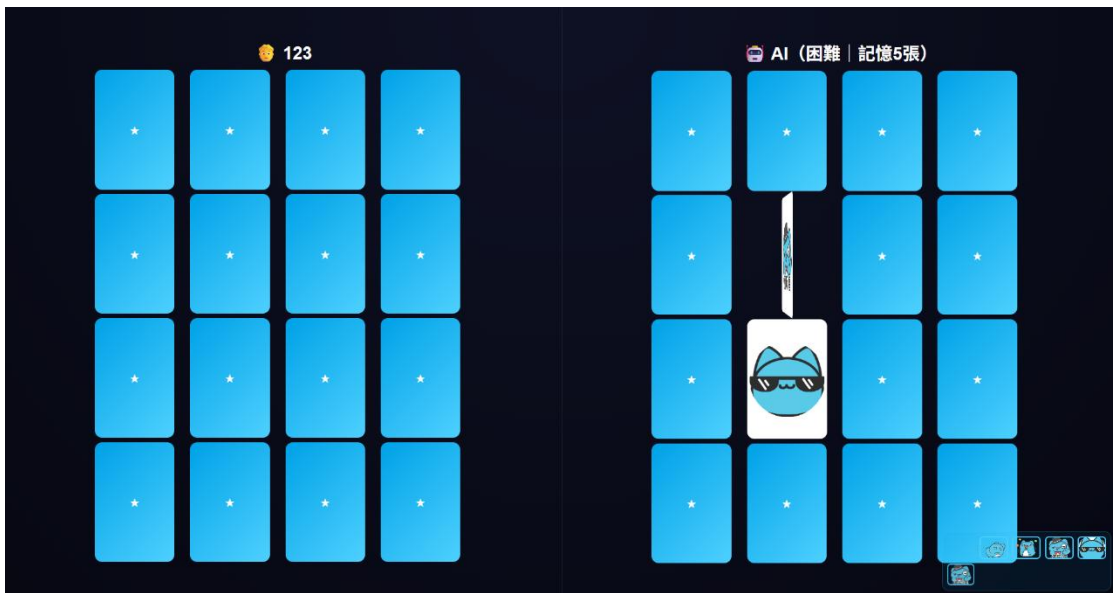
翻牌記憶大作戰



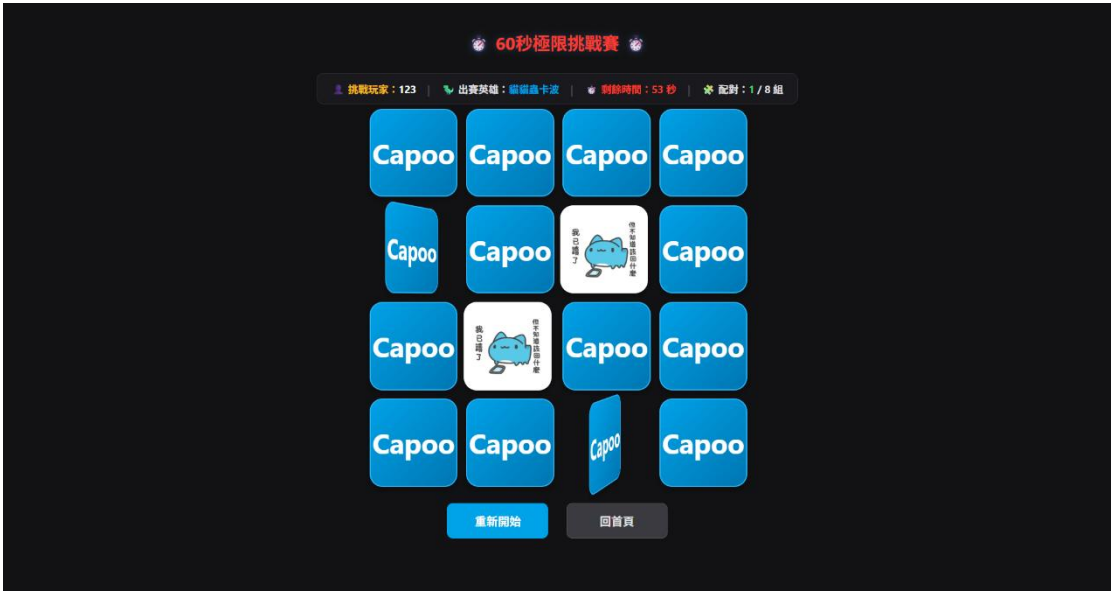
簡單模式(三張記憶點)



困難模式(五張記憶點)



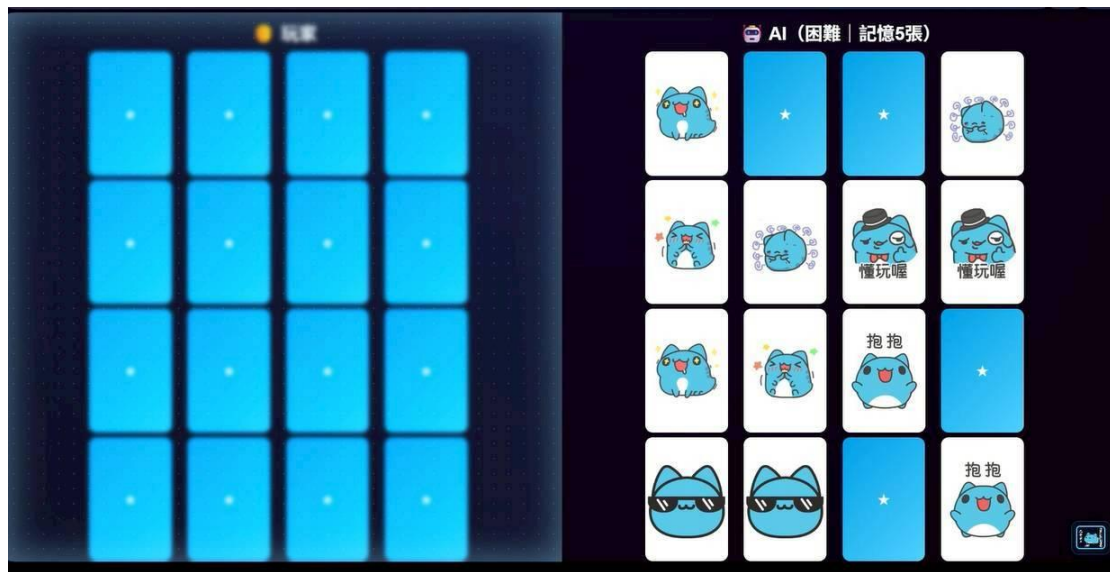
60 秒極限挑戰賽



COMBO 展示



凍結效果展示



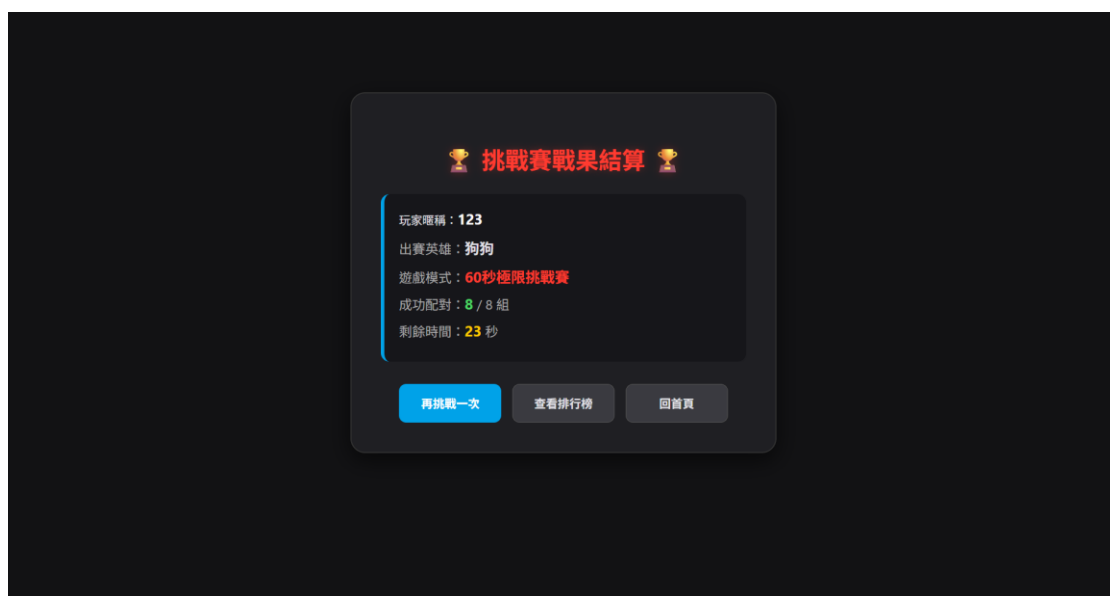
戰敗畫面



勝利畫面



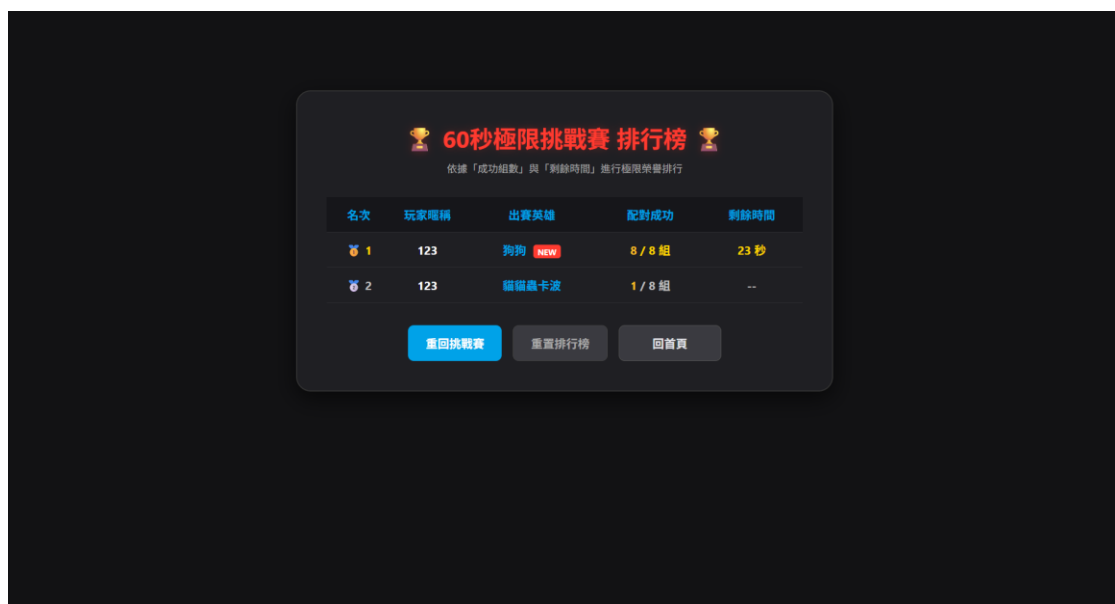
60 秒極限挑戰賽(結果畫面)



咖波群星圖鑑



排行榜



工作分配

黃筠晴:

story.html (角色介紹)

result.html (60 秒遊戲結果)

challenge.html (60 秒挑戰遊戲畫面)

leaderboard.html (60 秒遊戲排行榜)

陳思誠:

home.html(首頁)

challenge.html (60 秒挑戰遊戲畫面)

leaderboard.html (60 秒遊戲排行榜)

style.css

製作報告

製作報告文件說明檔

林展德:

index.html (遊戲模式及難度選擇)

game.html (AI 對戰遊戲畫面)

參考資料

- 貓貓蟲咖波官方網站 <https://capoo.com>
- 華山 1914 文創園區 <https://www.huashan1914.com>
- W3Schools HTML 教學 <https://www.w3schools.com>
- MDN Web Docs <https://developer.mozilla.org>
- jQuery 官方網站 <https://jquery.com>
- ChatGPT <https://chatgpt.com>